# セキュリティプロトコル

## SSL/TLS

学籍番号 (Student ID): 1280391
氏名 (Name): 細川夏風 (Hosokawa Natsuka)

高知工科大学 (Kochi Univ of Tech)

October 14, 2026

# 目次


1. 自己紹介
2. セキュリティプロトコル
   - セキュリティプロトコルについて
   - よりセキュアな通信
3. SSL/TLS の概要
4. SSL/TLS のプロトコル詳細
   - Record プロトコル
   - Handshake プロトコル
5. TLS
   - TLS1.3
   - DTLS

# 自己紹介

# 自己紹介

高知工科大学 情報学群 学士 3 年
所属研究室: シカール研究室

研究: SAGIN という新構想ネットワークの無線妨害の緩和手法

趣味: 家で作ったネットワーク環境をいじってアプリ開発とかハニーポッド作って遊んだり，音楽で Jazz をやったりもしてます．映画とかも好きで一番好きな映画は「インターステラー」です！




Figure: 好きな画家の絵を買えてニコニコの僕

# セキュリティプロトコル

# セキュリティプロトコル


1 自己紹介

2 セキュリティプロトコル
- セキュリティプロトコルについて
- よりセキュアな通信

3 SSL/TLS の概要

4 SSL/TLS のプロトコル詳細

5 TLS

# セキュリティプロトコルについて

## 通信の脅威

- 盗聴
- 改竄
- ハイジャック
- なりすまし
- 捏造
- 再送

## 通信を安全に行うための技術

- 公開鍵暗号方式
- 共通鍵暗号方式
- 鍵交換方式
- メッセージの改竄検知
- 認証プロトコル

セキュリティプロトコルはこれらの脅威に対して，右の技術を適切に組み合わせ用いる事によって CIA の確保を目指している．

# 利用場面とプロトコルの種類

Table: セキュリティプロトコルの利用場面と種類

| OSI 層 | 名称 | 概要 |
|---|---|---|
| アプリケーション層 | S/MIME | 電子メールの暗号化とディジタル署名のプロトコル |
| アプリケーション層 | SSH | 遠隔ログインのプロトコル |
| トランスポート層 | SSL/TLS | Web 通信などの様々な暗号化で利用されるプロトコル |
| ネットワーク層 | IPsec | IP パケット自体を保護対象とするプロトコル |
| データリンク層 | L2TP | 暗号器機能がないので IPsec などと併用される |

# セキュリティプロトコル


1. 自己紹介
2. セキュリティプロトコル
   - セキュリティプロトコルについて
   - よりセキュアな通信
3. SSL/TLS の概要
4. SSL/TLS のプロトコル詳細
5. TLS

# よりセキュアな通信

- **E2EE(End-to-End Encryption)**
  E2EE はメッセージアプリケーションなどにおいて，通信の当事者同士のみが閲覧可能である 2 点間の暗号方式のこと．
- **前方秘匿性**
  前方秘匿性は，もし仮に暗号が危殆化してしまった場合にそれまでに行われた通信の安全性を保証する性質のこと．
- **PCS(Post-Conpromise Security)**
  PCS は暗号方式が危殆化してもその後，ある程度は安全性を保証する概念のこと．ラチェット機構と呼ばれる仕組みで使用している鍵を変化させて通信を行うため，暗号が危殆化してもある程度の安全性は保証される．

# SSL/TLS の概要

# SSL/TLS の概要

- **SSL(Secure Socket Layer)**
  SSL は暗号化，認証，改竄検知などを行うセキュリティプロトコルである．これまでバージョンが 1, 2, 3 と更新されてきた．
- **TLS(Transport Layer Security)**
  SSL ver:3.0 以降の 1999 年に RFC2246 にて新たなセキュリティプロトコルである TLS が策定された．その後，UDP 上で TLS を実現する DTLS も策定割れた．

## SSL の使用について

1996 年以降の ver.3.0 から更新はされておらず，非推奨となっており，現在はほとんどのブラウザで使用されていない．それでもこの名前が用いられることが多いのは慣習の部分が大きいからだと思われる．

# SSL/TLS の構造

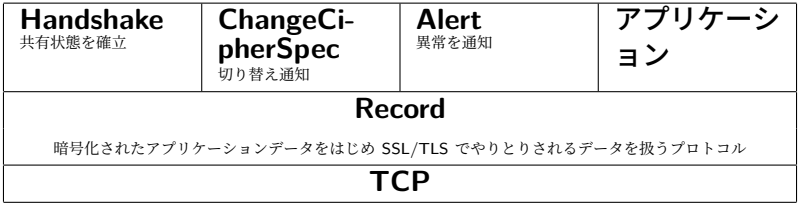



Figure: SSL/TLS の構造図

# SSL/TLS のプロトコル詳細

# SSL/TLS のプロトコル詳細


1. 自己紹介
2. セキュリティプロトコル
3. SSL/TLS の概要
4. SSL/TLS のプロトコル詳細
   - Record プロトコル
   - Handshake プロトコル
5. TLS

# Recode プロトコル

SSL/TLS のすべての通信は，送信するデータを最大 $2^{14}$ バイト以下のフラグメントに分ける．それらを個別に暗号化するなどして，受信者側に送信する．受信者はこの工程と逆の工程を行う．この手順の定義を行っているのが Record ぷろとこるである．
送信者は計算した MAC 値を添付し，暗号化し，ヘッダを追加する．この作成されたフラグメントのことを “record” という．実際にはこの “record” を送り合ってやり取りを行う．この際，シーケンス番号が “record” にはつけられているため，順序の検知が可能である．

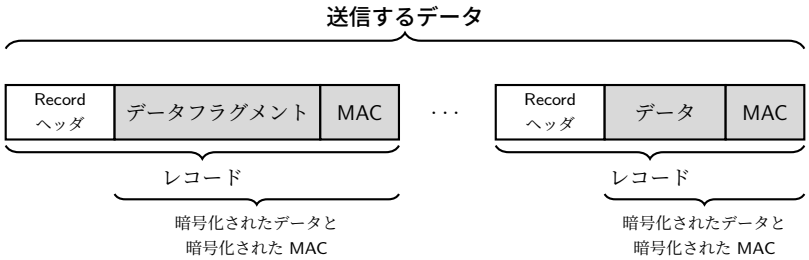



Figure: record のデータ処理

# Record プロトコル (Record ヘッダ)

Record ヘッダには，以下の 3 種類のフィールドがある．

## Record ヘッダのフィールド

- length
  フラグメント化する前の平文全体のサイズ．最大 $2^{14}$ を一塊として送受信する．
- SSL version
  通信に使用する SSL のバージョン．
- content type メッセージの種類を示したもの．4 種類のタイプがサポートされている．

# SSL/TLS のプロトコル詳細


1. 自己紹介
2. セキュリティプロトコル
3. SSL/TLS の概要
4. SSL/TLS のプロトコル詳細
   - Record プロトコル
   - Handshake プロトコル
5. TLS

# Handshake プロトコル

Handshake プロトコルの目的はお互いが等しく暗号アルゴリズムや暗号鍵などを等しく保持し，暗号通信に必要な共有状態を確立することである．暗号通信を行うために送受信者で合意して置かなければならない合意情報の組み合わせを “暗号スイート” という．

Handshake にはいくつかの種類がある．

- サーバー認証モード: RSA や DSS を用いてサーバを認証する方式
- クライアント認証モード: さらいにクライアントとサーバが相互に認証する方式

一般的にはサーバー認証モードが使われるが，よりセキュアなものにしたい場合はクライアント認証モードが使われることがある．

# Handshake プロトコル(メッセージ): サーバ認証モード

- **Hello Request**: クライアントは新たな方式での通信の合意のメッセージ
- **Client Hello**: クライアントが通信の開始を通知し，対応可能な暗号方式やランダム数を送信する．
- **Server Hello**: サーバーが使用する暗号方式などを決定して返信する．
- **Certificate**: サーバーのデジタル証明書を送信し，クライアントが検証する（**サーバー認証**）．
- **Server Key Exchange / Hello Done**: 鍵共有用のパラメータや，サーバー側のメッセージ送信終了を通知する．
- **Client Key Exchange**: クライアントが鍵共有データを送信し，共通の秘密鍵を生成する．
- **Change Cipher Spec & Finished**: お互いに暗号通信への切り替えとハンドシェイクの完了を確認し合う．

# Handshake プロトコル(メッセージ): クライアント認証モード

- **Client Hello**: クライアントが通信の開始を通知し，対応可能な暗号方式やランダム数を送信する．
- **Server Hello**: サーバーが使用する暗号方式などを決定して返信する．
- **Certificate**: サーバーのデジタル証明書を送信し，クライアントが検証する（**サーバー認証**）．
- **Server Key Exchange / Certificate Request / Hello Done**: 鍵共有パラメータや終了通知に加え，サーバーからクライアントへ証明書の提示を要求する．
- **Client Certificate / Client Key Exchange**: クライアント自身の証明書送信と鍵共有データの送信を行う．
- **Certificate Verify**: クライアントが秘密鍵の所持を証明する署名を送信する（**クライアント認証**）．
- **Change Cipher Spec & Finished**: お互いに暗号通信への切り替えとハンドシェイクの完了を確認し合う．

# TLS

# TLS


1. 自己紹介
2. セキュリティプロトコル
3. SSL/TLS の概要
4. SSL/TLS のプロトコル詳細
5. TLS
   - TLS1.3
   - DTLS

# TLS1.3 の概要

version1.2 から version1.3 になったことによっていくつかの改善があった.

## 変更点

- 利用可能な暗号スイートの変更
  - TLS_AES_128_CCM_SHA256: AEAD は AES_128_CCM, HKDF のハッシュ関数は SHA256
  - TLS_AES_128_CCM_8_SHA256: AEAD は AES_128_CCM_8, HKDF のハッシュ関数は SHA256
  - TLS_AES_128_GCM_SHA256: AEAD は AES_128_GCM, HKDF のハッシュ関数は SHA256
  - TLS_AES_256_GCM_SHA384: AEAD は AES_256_GCM, HKDF のハッシュ関数は SHA385
  - TLS_CHACHA20_POLY1305_SHA256: AEAD は CHACHA20_POLY1305, HKDF のハッシュ関数は SHA256
- Handshake の 1-RTT 化
  通信開始時に送る暗号スイートの候補リストからの返事をもらうだけの 1RTT 化に成功

# TLS


1. 自己紹介
2. セキュリティプロトコル
3. SSL/TLS の概要
4. SSL/TLS のプロトコル詳細
5. TLS
   - TLS1.3
   - DTLS

# DTLS の概要

TLS は TCP を想定して作れられているプロトコルであるため，そのままに UDP 上で動かすことは難しい．その問題はメッセージを単位フラグメントあたりでは復号ができないため損失や順序に問題がなく届く必要がある．
そのために以下のような修正が行われている．

## 修正内容

- Handshake の修正: 確認用返答メッセージとタイマーによる返答確認時間の導入
- record の修正:
  - 複数のコネクション識別のための epoch フィールドの追加
  - シーケンス番号の確認のための sequence_number の追加

# セキュリティ対策

**再送攻撃対策**
DTLS には，シーケンス番号があるため，再送攻撃を防ぐことが可能である．

**DoS 攻撃対策**
この問題の解決のためには IPsec でも用いられるステートレスクッキー (Stateless cookie) という技術を用いている．最初の通信にステートレスクッキーを生成し，それを添付して通信を行う．今後はそれを認証することによって通信を維持していく．